

Assignment 2

Regression: MLP & Deep Neural Network (DNN)

SCANIA Component X Dataset - Predictive Maintenance

Course: AI & Neural Networks

University: Astana IT University (AITU)

Date: March 2026

1. Introduction

This report presents the implementation and evaluation of two fully connected neural network architectures - a Multi-Layer Perceptron (MLP) and a Deep Neural Network (DNN) - for a regression task on the SCANIA Component X dataset. The goal is to predict the time-to-event (`length_of_study_time_step`), a continuous variable representing how long a vehicles component remains in operation before potential failure.

The processed dataset from Assignment 1 (50,000 samples, 125 features after preprocessing) serves as input. We compare the two architectures on RMSE and MAE metrics and draw conclusions about the applicability of fully connected networks to time series data.

2. Data Preparation

2.1 Dataset

The processed dataset (`processed_data.csv`) from Assignment 1 was loaded. It contains 50,000 observations sampled from the full SCANIA dataset (559,091 rows). After excluding non-informative columns (`vehicle_id`, `time_step`, `class_label`, `time_of_day`), we retained 125 features including sensor readings, vehicle specifications, and engineered features (lag, rolling mean, rate of change).

2.2 Target Variable

The regression target is `length_of_study_time_step` - a continuous value representing the total operational time of each vehicle in the study. Statistics: Mean = 297.84, Std = 96.67, Min = 74.0, Max = 510.0.

2.3 Train/Validation/Test Split

The data was split in a 70/15/15 ratio strictly preserving chronological order (no shuffling), as required for time series data to prevent data leakage from future observations:

- Train: 35,000 samples (70%)
- Validation: 7,500 samples (15%)
- Test: 7,500 samples (15%)

2.4 Feature Scaling

`StandardScaler` was applied to all features. Critically, the scaler was fit only on the training set and then used to transform the validation and test sets, preventing data leakage.

3. MLP Model (Multi-Layer Perceptron)

3.1 Architecture

A shallow fully connected network with 3 hidden layers:

Input (125) -> Dense(64, ReLU) -> Dense(32, ReLU) -> Dense(16, ReLU) -> Dense(1, Linear)

Total parameters: 10,689

3.2 Training Configuration

- Optimizer: Adam (learning rate = 0.001)

- Loss function: Mean Squared Error (MSE)
- Batch size: 64
- Max epochs: 100
- EarlyStopping: patience=10, restore_best_weights=True

3.3 Training Results

The MLP training stopped early at epoch 40 (best weights from epoch 30) with a best validation loss of 3,173.60. The training loss decreased steadily from ~23,000 to ~2,750, indicating effective learning.

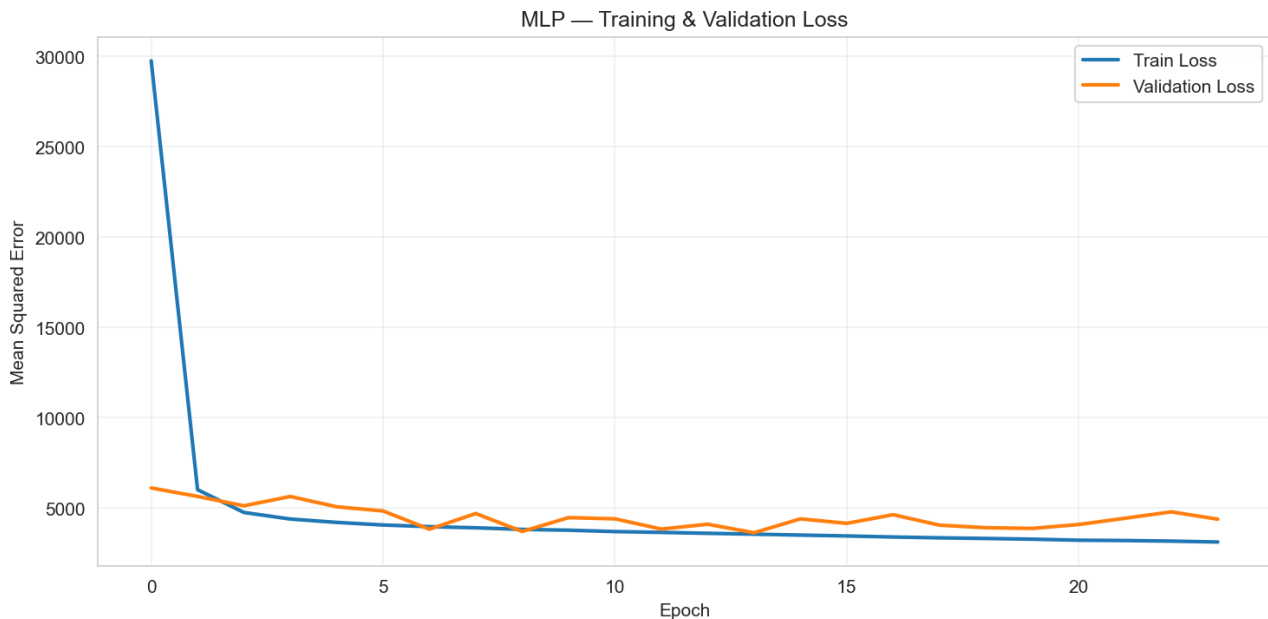


Figure 1: MLP Training & Validation Loss

4. Deep Neural Network (DNN)

4.1 Architecture

A deeper network with 5 hidden layers to test whether increased depth improves performance:

Input (125) -> Dense(128, ReLU) -> Dense(64, ReLU) -> Dense(32, ReLU) -> Dense(16, ReLU) -> Dense(8, ReLU) -> Dense(1, Linear)

Total parameters: 27,137

4.2 Training Results

The DNN stopped at epoch 34 (best weights from epoch 24) with a best validation loss of 2,361.80 - significantly better than the MLPs 3,173.60. The deeper architecture converged to a lower loss faster despite having more parameters.

Assignment 2: MLP & DNN Regression - SCANIA Component X



Figure 2: DNN Loss (left) and MLP vs DNN Validation Loss Comparison (right)

5. Quality Evaluation

5.1 Metrics Comparison

RMSE and MAE were computed on all three splits for both models:

Model	Split	RMSE	MAE
MLP	Train	53.69	40.02
MLP	Validation	56.33	45.31
MLP	Test	68.82	55.89
DNN	Train	52.28	38.32
DNN	Validation	48.60	38.53
DNN	Test	65.93	53.62

The DNN outperforms the MLP on all splits. On the test set, the DNN achieves an RMSE of 65.93 versus 68.82 for the MLP - a 4.2% improvement. The MAE shows a similar pattern (53.62 vs 55.89).

Note on MSE scale: The raw MSE values appear large (2,000-6,000) because the target variable ranges from 74 to 510. An RMSE of ~66 relative to a standard deviation of ~97 indicates the models explain a meaningful portion of the target variance.

5.2 Actual vs Predicted - Scatter Plot

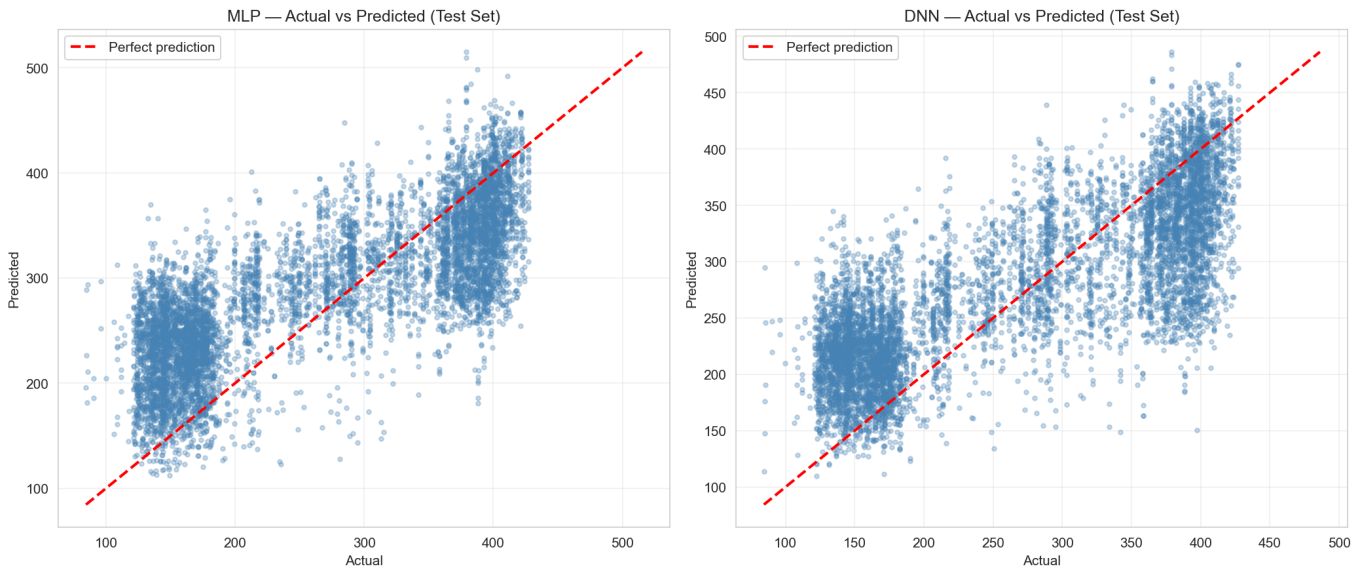


Figure 3: Actual vs Predicted on Test Set (MLP left, DNN right)

5.3 Actual vs Predicted - Line Plot

Assignment 2: MLP & DNN Regression - SCANIA Component X

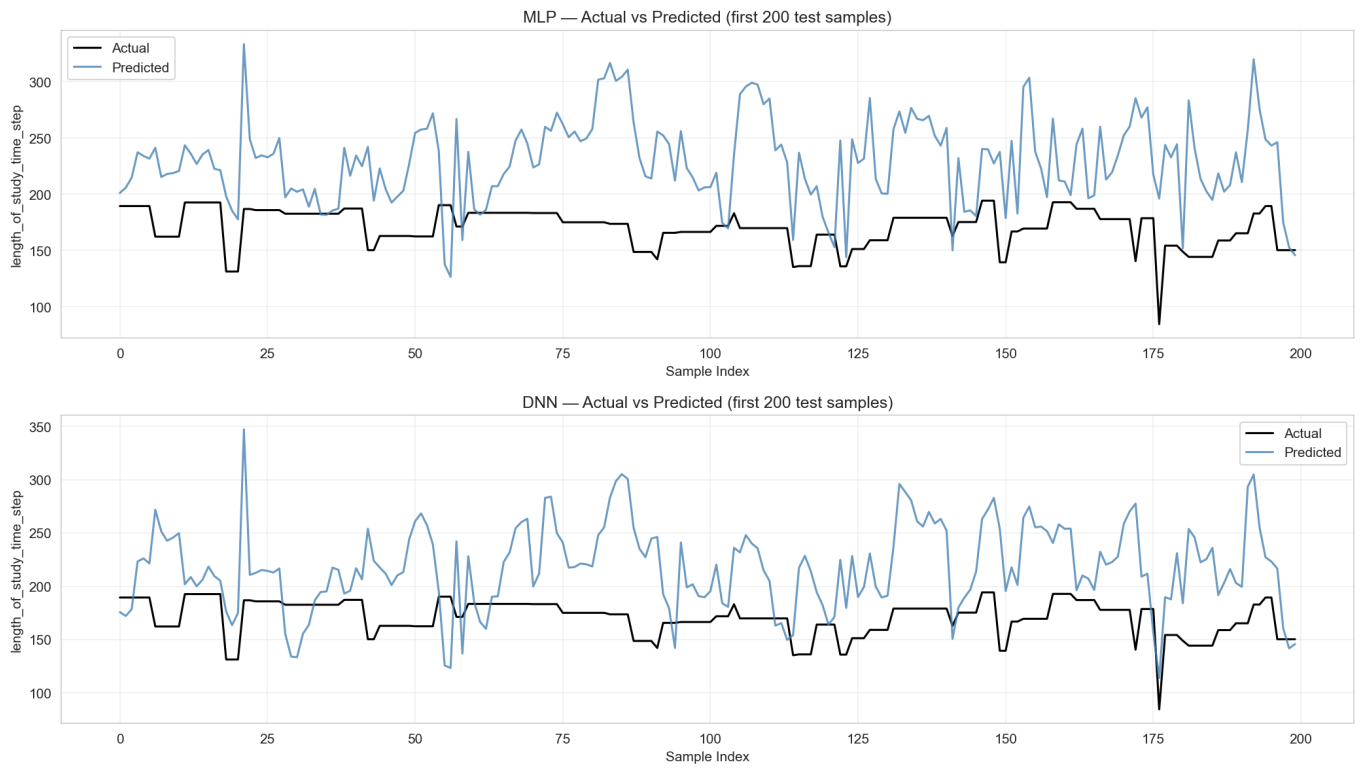


Figure 4: Actual vs Predicted for first 200 test samples (MLP top, DNN bottom)

6. Model Comparison & Discussion

MLP (3 hidden layers, 10,689 params)

- Faster training (~0.6ms/step)
- Stopped at epoch 40, best at epoch 30
- Higher test error (RMSE: 68.82)
- More stable but potentially underfitting

DNN (5 hidden layers, 27,137 params)

- Slightly slower training (~1ms/step)
- Stopped at epoch 34, best at epoch 24
- Lower test error (RMSE: 65.93)
- Greater capacity captured more complex patterns

Impact of Depth

The deeper DNN architecture provided a modest but consistent improvement over the MLP. The additional layers (128 and 8 neurons) allowed the network to learn more complex non-linear mappings between sensor readings and time-to-event. However, the improvement is not dramatic (~4%), suggesting that the relationship captured by the shallower network already accounts for most learnable patterns in the tabular features.

7. Conclusions

1. Both MLP and DNN successfully learned to predict time-to-event from sensor data, with the DNN achieving slightly better performance (Test RMSE: 65.93 vs 68.82).
2. EarlyStopping with `restore_best_weights` was essential - both models showed signs of overfitting (validation loss oscillating while training loss decreased), and early stopping prevented degradation.
3. Chronological splitting (no shuffling) is critical for time series to avoid data leakage. The higher test error compared to validation error suggests the model encounters distribution shift in later time periods, which is expected in temporal data.
4. Fully connected networks treat each observation independently and cannot model temporal dependencies between consecutive readings. For improved performance on this time series task, recurrent architectures (RNN, LSTM) or Transformers would be more appropriate - these will be explored in subsequent assignments.
5. The best model (DNN) was saved as `mlp_best.h5` for use in future assignments.